

Teaching a Coding Agent to Design DNA Origami: What Worked, What Didn't, and What It Took to Get There

Daniel Fu

March 2026

Abstract

We report on our experience using a general-purpose coding agent (Claude Code) to design DNA origami structures in cadnano. We describe two approaches — an embedded tool-using LLM that failed, and a coding agent with source code access that succeeded — and trace the path from initial failure to a working parametric pipeline for a 2-layer rectangular origami with a cavity. The key finding is that the agent's bottleneck was not code generation but domain knowledge: each class of design error required explicit correction from the human designer before the agent could proceed. We detail the specific errors, the domain knowledge that unblocked each one, and the capabilities that emerged after unblocking.

1. The Problem

Modifying a DNA origami design is slow. When a cavity is too narrow by 5 nm, the designer must re-route the scaffold, re-place staples, re-apply edge treatments, and re-verify the structure. This process takes hours and must be repeated for every design iteration. We asked: can an AI agent handle this redesign automatically?

2. First Attempt: Embed an LLM Inside cadnano

We built an agent dialog into the cadnano GUI (Figure 1a). The user types a natural language command; an LLM parses the intent and calls Python methods that modify the design. We implemented 85 methods covering helix creation, strand manipulation, crossover placement, and staple breaking.

This failed on anything beyond single-step operations. Scaffold routing — the task of placing crossovers to create one continuous scaffold loop — requires reasoning about helix parity, crossover position tables, and routing topology simultaneously. Even GPT-class models achieved 0% success. We attempted reinforcement learning with a design verifier as reward signal, but the local model could not discover correct action sequences through exploration.

What we learned: The API encodes necessary but insufficient domain knowledge. The model must compose methods into correct sequences without understanding why each step matters.

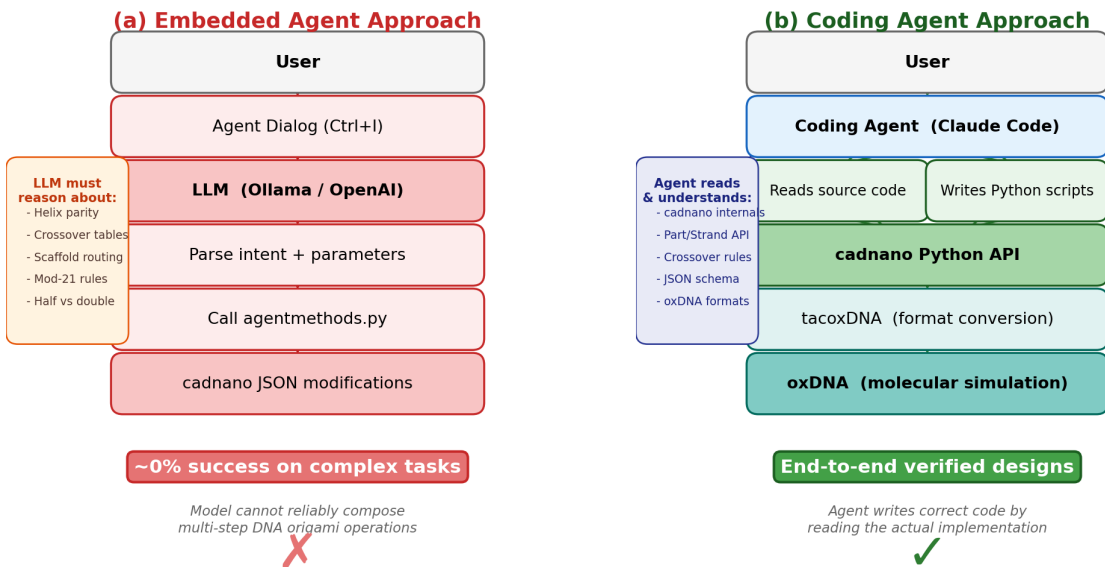


Figure 1: **(a)** The embedded agent approach: an LLM parses natural language and calls a fixed 85-method API. Failed on multi-step tasks (~0% success on scaffold routing). **(b)** The coding agent approach: reads cadnano source code, writes Python scripts, integrates tacoxDNA and oxDNA. Produces end-to-end verified designs.

3. Second Attempt: A Coding Agent with Source Code Access

We switched to a coding agent (Claude Code) that reads cadnano’s source code and writes Python scripts. This immediately solved several problems the embedded agent could not. When `createXover` broke scaffold continuity by splitting strands, the agent found `setConnection3p/5p` by reading the strand model internals. When the autobreak plugin failed to import, the agent traced the module system and used `importlib` to bypass a legacy `__init__.py`. The agent also integrated cadnano with tacoxDNA and oxDNA, creating an end-to-end design-to-verification pipeline.

First success: A rectangular origami with the m13mp18 scaffold (7,249 nt). 24 helices, serpentine routing, autoStaple, autoBreak, twist correction, tacoxDNA conversion, oxDNA relaxation (Figure 2). This is trivial for a human designer — but it proved the pipeline worked.

4. Scaling Up: Where Domain Knowledge Became the Bottleneck

We then attempted a 2-layer rectangle with a rectangular cavity using the p8064 scaffold (8,064 nt). This is where the agent’s limitations became clear. Each step forward required the human designer to diagnose and correct a specific class of error.

Error 1: Wrong grid layout. The agent built a 4×7 grid. A 2-layer rectangle should be 2×12 (2 grid rows = 1 honeycomb row on the lattice). The agent did not know this mapping. *Unblocked by:* the PI explaining the grid-to-layer correspondence.

Error 2: Serpentine routing instead of dense crossovers. The agent routed the scaffold with one crossover per helix pair. Real DNA origami uses multiple double crossovers per pair — the

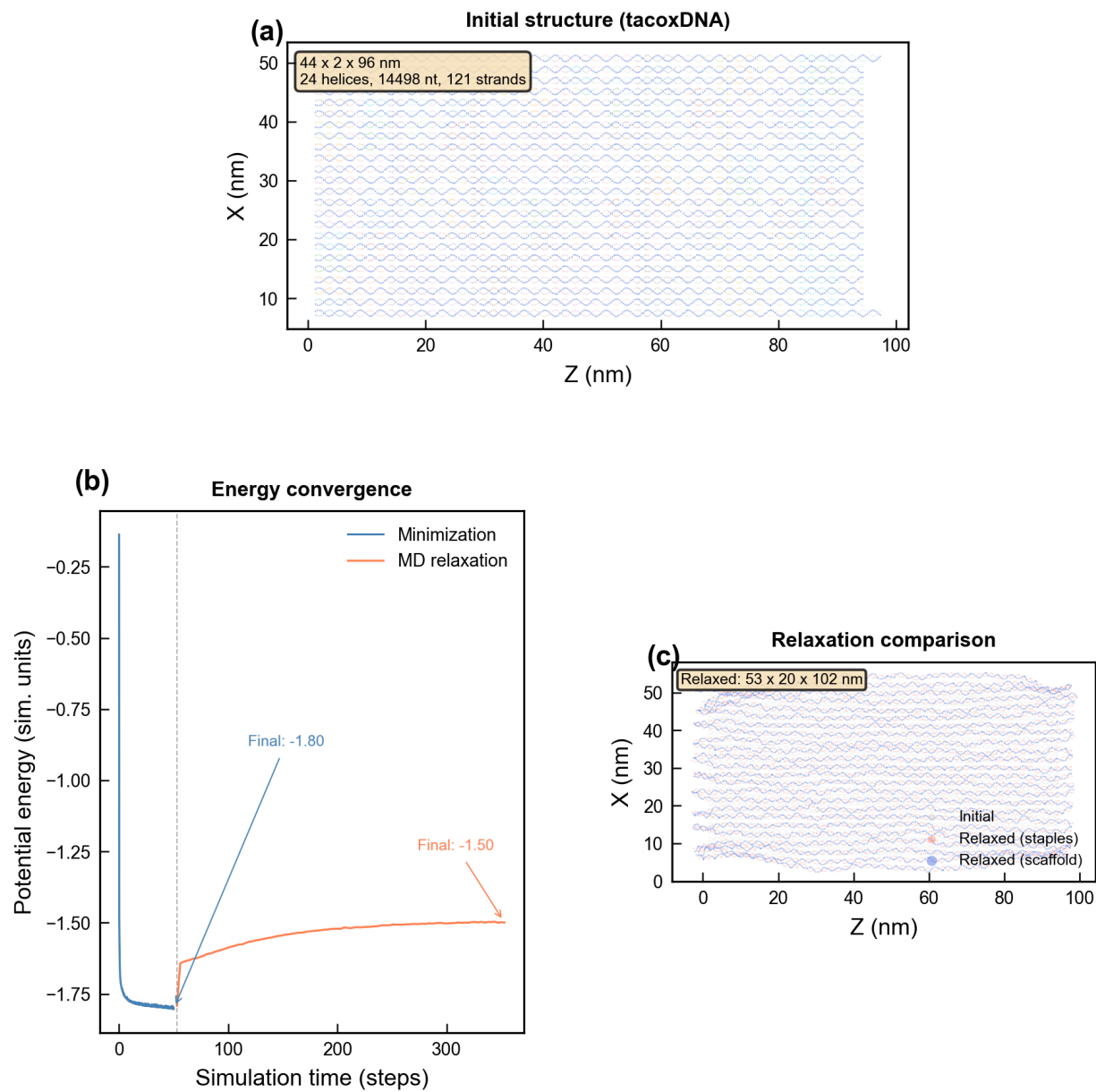


Figure 2: Proof of concept: rectangular DNA origami with m13mp18 scaffold. **(a)** Initial 3D structure from tacoxDNA conversion (side view). **(b)** Energy convergence during oxDNA minimization and relaxation. **(c)** Relaxation comparison showing structural dimensions.

scaffold weaves back and forth. *Unblocked by:* the PI providing a hand-designed template showing the correct crossover pattern (Figure 3).

Error 3: Cavity in the wrong dimension. The agent placed the cavity by removing helices from the XY cross-section. The cavity should be in the Z direction — a gap along the helix length axis. *Unblocked by:* the PI specifying the correct orientation.

Error 4: Scaffold/staple confusion. When building from scratch, the agent placed scaffold strands on the staple strand set. *Unblocked by:* the PI inspecting the design in cadnano.

Error 5: Scaling broke file format. When scaling the template, the agent left stray staple entries with broken crossover references. cadnano crashed on load. *Unblocked by:* tracing the crash to the legacy decoder and identifying the broken references.

Error 6: Cavity gaps filled incorrectly. The agent filled the cavity region with continuous scaffold, destroying the cavity. *Unblocked by:* the PI manually clearing the cavity positions and returning the corrected file.

Error 7: Crossover misalignment. The scaling logic moved crossover positions independently per helix, creating skewed crossovers. Both sides must be at the same base pair index. *Unblocked by:* the PI aligning the crossovers.

Error 8: Cavity edges not aligned between layers. The cavity right edge was at different positions in each layer. *Resolved by:* the agent computing valid crossover positions for each honeycomb direction and selecting the closest match (2 bp offset — the minimum the lattice geometry allows).

5. What the Agent Could Do After Unblocking

After receiving the PI’s template, the agent developed a 4-step scaling process, validated step-by-step (Figure 4):

1. **Remove staples** (keep scaffold only). Verified loads in cadnano.
2. **Extend helix arrays** and shift right-side crossovers. Verified loads.
3. **Move midseam crossovers** to the center of the extended helix. Verified loads.
4. **Expand cavity** by moving boundary crossovers, aligning between layers. Verified loads.

The final result: a 2×12 rectangular origami with an aligned cavity, 1 continuous scaffold loop (7,688 bp), scaled from the PI’s 252 bp template to 420 bp helices. Each step produced a valid cadnano JSON.

6. What This Tells Us

The agent is not a designer. It is a fast, tireless assistant that can manipulate files, trace code, and automate repetitive operations — but it cannot independently determine the correct design. Every non-trivial design decision required human domain knowledge.

The value is in what happens *after* the human provides the knowledge. Once shown the correct pattern, the agent could scale it, modify it, and verify each step in seconds. The 4-step scaling process that took an afternoon of back-and-forth would take days if done entirely by hand for each new scaffold length.

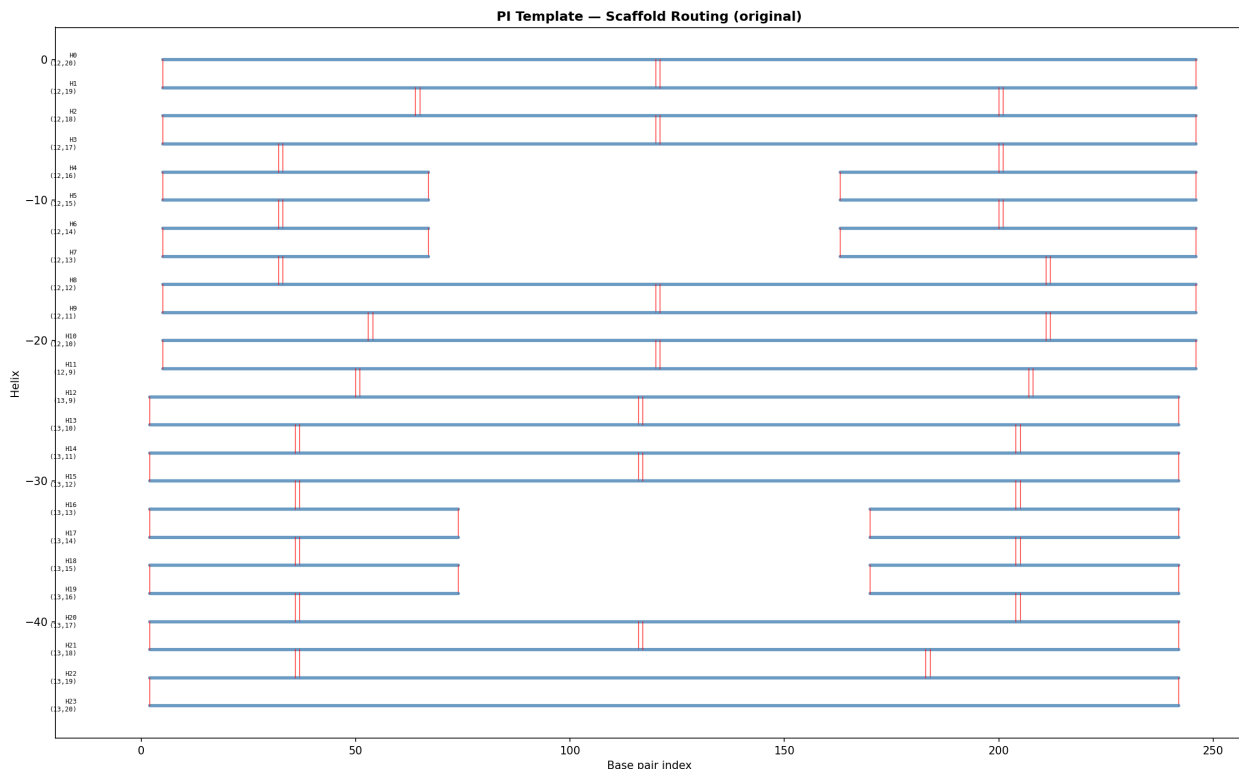


Figure 3: The PI's hand-designed template (`2x12_rectangle_cavity.json`) showing correct scaffold routing with dense crossovers. Each horizontal blue line is one helix; red vertical lines are crossovers between helix pairs. Cavity helices (H4-H7, H16-H19) have gaps in the middle. This template was the key domain knowledge that unblocked the agent's progress.

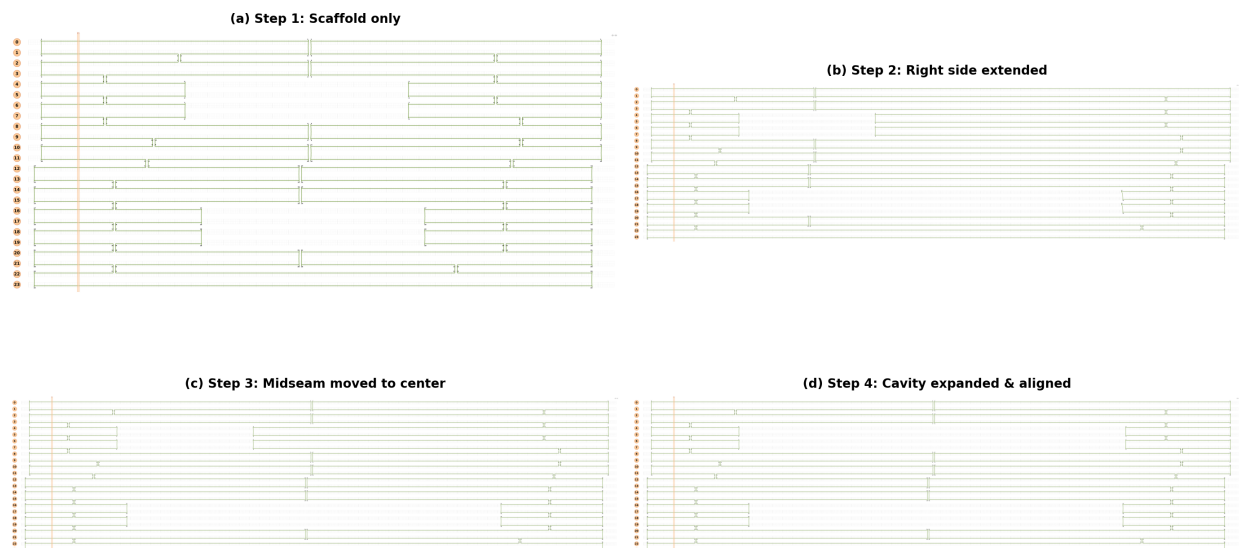


Figure 4: The 4-step scaling process from the PI's template (252 bp) to the p8064 target (420 bp). **(a)** Step 1: scaffold only (staples removed). **(b)** Step 2: right side extended by 168 bp. **(c)** Step 3: midseam crossovers moved to center. **(d)** Step 4: cavity expanded with edges aligned between layers.

For DNA nanotechnologists: A coding agent is most useful not as an autonomous designer, but as a parametric design tool. You design the template; the agent scales, modifies, and verifies. The bottleneck shifts from clicking in cadnano to communicating design intent clearly.

What remains difficult: The agent cannot discover design conventions from source code alone. Honeycomb layer counting, dense crossover patterns, cavity orientation, crossover alignment rules — these are domain knowledge that must be taught explicitly. Each new design feature risks a new class of error that requires human diagnosis.